

Creating Custom Forms Widget

 Information last updated 22 sept 2025

SME	Jan Rusch
EPIC	CIC Custom User Interface and Custom Development

- 1  Instructions
- 1.1  Form Widget Development Process Flow

1.2  Prerequisites

1.3  Step 1: Create a New Git Branch

1.4  Step 2: Generate Your Plugin

1.5  Step 3: Generate a Form Widget

1.6 This creates a widget that can be used in forms just like any standard form field!

1.7  Step 4: Enhance Your Widget Component (Recommended)

1.7.1 4.1 Create Template and Style Files

1.7.2 4.2 Update Component File

1.8  Step 5: Configure in Studio Modeling

1.8.1 5.1 Create Form Widget in Studio Modeling

1.8.2 5.2 Configure the Widget Type

1.9  Step 6: Test Your Widget

1.10  Video: How to Create a Custom Form Widget

1.11  Next Steps

1.12  Questions?
- 2  Related articles

Instructions

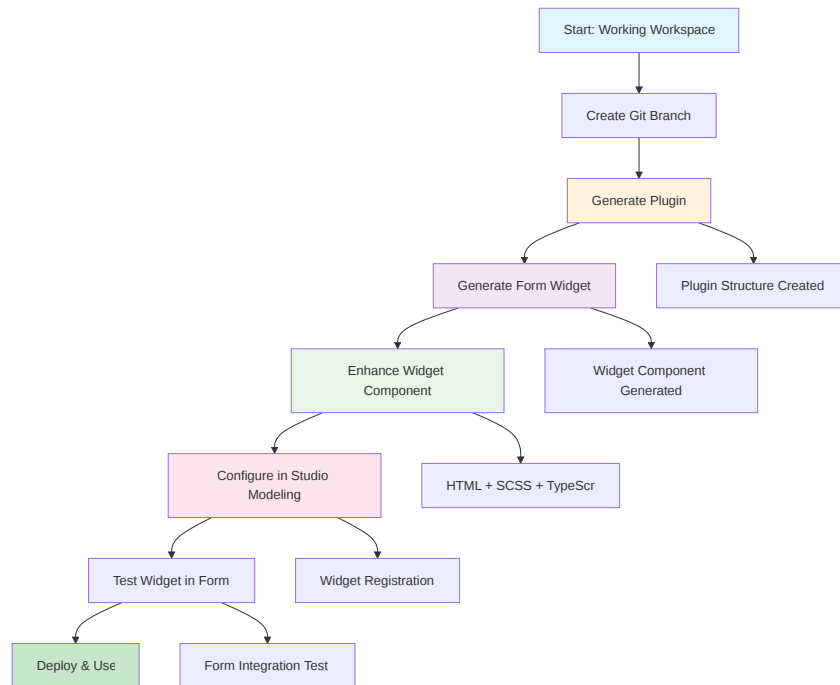
Hey developers! Welcome back to another Hyland Experience tutorial. Today, I'm going to show you how to create custom form widgets that can be used in your Hyland Experience forms. These widgets give you the power to create unique, interactive form elements that go beyond the standard text fields and dropdowns.

What you'll learn today:

- How to create custom form widgets using the Hyland Extend framework
- How to integrate widgets with form data and validation
- How to configure widgets in Studio Modeling
- Advanced techniques for widget development

Let's build some powerful form components!

Form Widget Development Process Flow



Prerequisites

Before we start, make sure you have:

- **A working Hyland Experience Workspace Custom UI.**
- **Node.js 18+ and npm** installed
- **A code editor** (VS Code recommended)
- **Access to Studio Modeling** for widget configuration
- **Basic understanding of Angular components**

Step 1: Create a New Git Branch

IMPORTANT: Before we start generating code, create a new branch! The plugin generation commands create and modify many files, so having a clean branch makes it easy to start over if needed.

```
1 | git checkout -b feature/custom-form-widgets
```

 This gives you a safety net to experiment with different widget names and configurations.

Step 2: Generate Your Plugin

First, we need to create a plugin to contain our form widgets:

```
1 | npx nx generate @hyland/extend:plugin --name form-widget-library --author "Your Name" --addTranslations true
```

Parameters explained:

- `--name form-widget-library` : Your plugin identifier
- `--author "Your Name"` : Your name (for documentation purposes)
- `--addTranslations true` : Enables internationalization support

✓ Choose a descriptive name that reflects your widget library's purpose. This will be the foundation for all your custom form widgets.

Step 3: Generate a Form Widget

Now let's create our first custom form widget:

```
1 npx nx generate @hyland/extend:form-widget --pluginName form-widget-library --formWidgetName repeating-section
```

Parameters explained:

- `--pluginName form-widget-library` : The name of your plugin (without "plugins-" prefix)
- `--formWidgetName repeating-section` : The name of your widget (will be converted to Angular naming conventions)

What gets generated:

- `repeating-section.component.ts` - Your form widget component
- Proper integration with the plugin framework
- Base widget functionality extending `WidgetComponent`

This creates a widget that can be used in forms just like any standard form field!

Step 4: Enhance Your Widget Component (Recommended)

The generated component only creates a TypeScript file. For better development experience, let's add HTML templates and SCSS files:

4.1 Create Template and Style Files

Navigate to your widget directory and create these files:

- `repeating-section.component.html`

✓ Create : repeating-section.component.html :

```
1 <div class="repeating-section-widget {{field.className}}" [class.adf-readonly]="field.readOnly">
2   <label class="adf-label" [attr.for]="field.id">
3     {{field.name}}
4     <span class="adf-asterisk" *ngIf="isRequired()">*</span>
5   </label>
6
7   <div class="repeating-section-container">
8     <!-- Add Button -->
9     <button
10       mat-raised-button
11       color="primary"
12       (click)="addSection()"
13       *ngIf="!field.readOnly"
14     >
15       Add Section
16     </button>
17
18     <!-- Sections List -->
19     <div class="section" *ngFor="let section of sections; let i = index">
20       <div class="section-header">
21         <span>Section {{ i + 1 }}{{ section.name ? ': ' + section.name : '' }}</span>
22         <button
23           mat-icon-button
24           (click)="removeSection(i)"
25           *ngIf="!field.readOnly"
26         >
27           <mat-icon>delete</mat-icon>
28         </button>
29       </div>
30       <div class="section-content">
31         <!-- Display section data -->
32         <div class="section-data" *ngIf="section.name || section.id">
```

```

33         <p *ngIf="section.name"><strong>Name:</strong> {{ section.name }}</p>
34         <p *ngIf="section.id"><strong>ID:</strong> {{ section.id }}</p>
35     </div>
36     <!-- Form fields go here -->
37     <ng-content></ng-content>
38 </div>
39 </div>
40
41 <!-- Empty State -->
42 <div class="empty-state" *ngIf="sections.length === 0">
43     <p>No sections added yet</p>
44 </div>
45 </div>
46 </div>

```

• repeating-section.component.scss

▼ Create : repeating-section.component.scss

```

1  .repeating-section-widget {
2      margin-bottom: 16px;
3      padding: 16px;
4      border: 1px solid #ddd;
5      border-radius: 4px;
6      background: #f9f9f9;
7
8      .adf-label {
9          display: block;
10         margin-bottom: 12px;
11         font-weight: 500;
12     }
13
14     .repeating-section-container {
15         .section {
16             margin: 12px 0;
17             border: 1px solid #ccc;
18             border-radius: 4px;
19             background: white;
20
21             .section-header {
22                 display: flex;
23                 justify-content: space-between;
24                 align-items: center;
25                 padding: 12px;
26                 background: #f5f5f5;
27                 border-bottom: 1px solid #ccc;
28             }
29
30             .section-content {
31                 padding: 12px;
32             }
33         }
34     }
35
36     .empty-state {
37         text-align: center;
38         padding: 20px;
39         color: #666;
40     }
41 }
42

```

4.2 Update Component File

▼ Update repeating-section.component.ts:

```

1  import { FormService, WidgetComponent } from '@alfresco/adf-core';
2  import { Component } from '@angular/core';
3  import { NgFor, NgIf } from '@angular/common';
4  import { MatButtonModule } from '@angular/material/button';
5  import { MatIconModule } from '@angular/material/icon';
6
7  @Component({
8      templateUrl: './repeating-section.component.html',
9      styleUrls: ['./repeating-section.component.scss'],
10     selector: 'form-widget-library-repeating-section',
11     standalone: true,
12     imports: [NgFor, NgIf, MatButtonModule, MatIconModule],
13 })
14 export class RepeatingSectionComponent extends WidgetComponent {

```

```

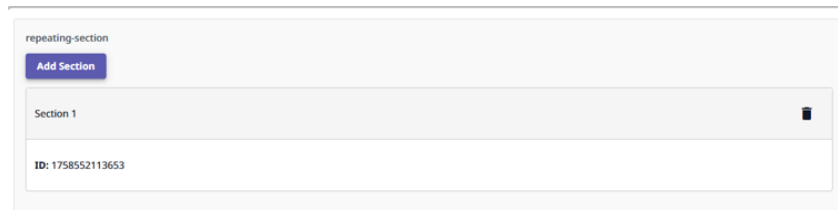
15 sections: any[] = [];
16
17 constructor(public override formService: FormService) {
18     super(formService);
19 }
20
21
22 ngOnInit(): void {
23     // Initialize sections from field value
24     if (this.field.value) {
25         if (Array.isArray(this.field.value)) {
26             // Parse each array entry in case they are stringified JSON
27             this.sections = this.field.value.map(entry => this.parseEntry(entry));
28         } else if (typeof this.field.value === 'string') {
29             try {
30                 // Try to parse as JSON if it's a string
31                 const parsed = JSON.parse(this.field.value);
32                 if (Array.isArray(parsed)) {
33                     // Parse each array entry in case they are stringified JSON
34                     this.sections = parsed.map(entry => this.parseEntry(entry));
35                 } else {
36                     this.sections = [this.parseEntry(parsed)];
37                 }
38             } catch (error) {
39                 console.warn('Failed to parse field value as JSON:', this.field.value);
40                 this.sections = [];
41             }
42         } else {
43             // Single object, wrap in array
44             this.sections = [this.parseEntry(this.field.value)];
45         }
46     }
47 }
48
49 private parseEntry(entry: any): any {
50     if (typeof entry === 'string') {
51         try {
52             // Try to parse the entry as JSON if it's a string
53             return JSON.parse(entry);
54         } catch (error) {
55             console.warn('Failed to parse entry as JSON:', entry);
56             return entry; // Return as-is if parsing fails
57         }
58     }
59     return entry; // Return as-is if not a string
60 }
61
62 addSection(): void {
63     const newSection = {
64         id: Date.now(), // Simple unique ID
65         name: ""
66     };
67
68     this.sections.push(newSection);
69     this.updateFieldValue();
70 }
71
72 removeSection(index: number): void {
73     this.sections.splice(index, 1);
74     this.updateFieldValue();
75 }
76
77 private updateFieldValue(): void {
78     this.field.value = this.sections;
79     this.fieldChanged.emit(this.field);
80 }
81 }

```

repeating-section

Add Section

No sections added yet



⚙️ Step 5: Configure in Studio Modeling

5.1 Create Form Widget in Studio Modeling

1. **Navigate to Studio Modeling** and go to your project
2. **Click on Form Widgets** in the left navigation
3. **Click the “More” Button** to create a new form widget
4. **Give it a name** (e.g., "repeating-section")
5. **Choose a value type** (in this example Array)

5.2 Configure the Widget Type

1. **Switch to JSON Editor** (click the eyeball icon)
2. **Find the "type" field** and note its value

```
1 {
2   "id": "821e2e91-c094-4cbe-b0c2-ed1b38e244b4",
3   "name": "repeating-section",
4   "key": "repeating-section-86vnh",
5   "description": "",
6   "type": "repeating-section",
7   "isCustomType": true,
8   "valueType": "array",
9   "icon": "data:image/png;base64,iVBORw0KGgoAAAANSUhEUgAAABgAAAAACAAAAAAYCAAAA",
10  "className": ""
11 }
```

3. Update your Angular module to match this type

- `libs\plugins\form-widget-library\src\lib\form-widgets\repeating-section\repeating-section.module.ts`

```
@NgModule({
  imports: [RepeatingSectionComponent],
})
export class RepeatingSectionModule {
  constructor(formService: FormRenderingService) {
    formService.register({
      'repeating-section': () => RepeatingSectionComponent,
    });
  }
}
```



- Or vice versa :)

🚀 Step 6: Test Your Widget

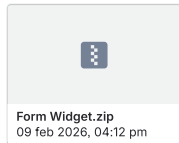
1. **Start your development server:**

```
1 | npm start workspace-hxp
```

2. **Create a form** in Studio Modeling and add your custom widget
 3. **Deploy and test** the form to see your widget in action
-

📺 Video: How to Create a Custom Form Widget

▶ [Panopto](#)



🔧 Next Steps

Now that you have a working form widget, you can:

- **Create more complex widgets** with advanced functionality
 - **Add data validation** and error handling
 - **Integrate with external APIs** for dynamic content
 - **Deploy your widgets** (see our [Packaging a Custom UI](#) guide)
-

💬 Questions?

Reach out to the team! We're here to help you! 🚀

Thanks, and happy coding!

📄 Related articles